ECE 385

SPRING 2025

Experiment 2

Data Storage

Artha Amanda Sidauruk
(3220300399)
Cristina Natalia Lumban Tobing
(3220300398)
Friday, Feb 26, 2025. D-231
TA: Qiyang Wu

1 INTRODUCTION

The purpose of this lab is to design a Shift Register Memory, specifically a simple 2-bit, four-word shift-register storage unit. The memory will consist of four words, where each of them containing 2 bits. Using serial input and output methods, the designed circuit performs several operations, namely, load, read, write, and do nothing. Below is a big overview of how to build the data storage with TTL.

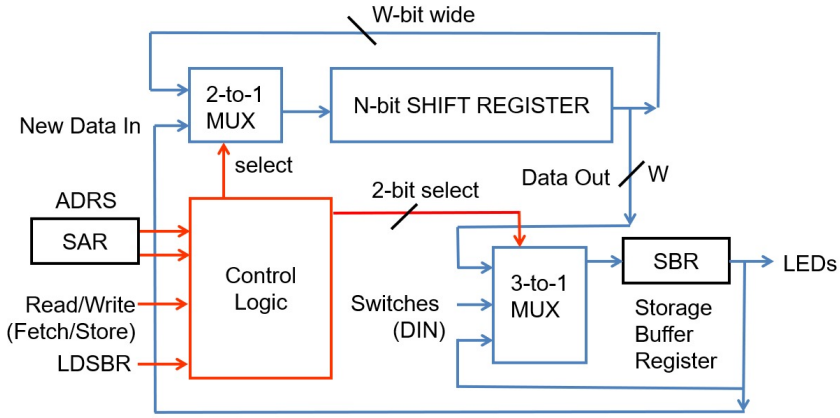

Figure 1: Big Overview

Based on the diagram above, the control logic, composed of a counter and comparator, plays a role in deciding the read/write operations. Along with this, a 3-to-1 MUX is also used to determine the fetch, store, or load operations. For data shifting and storage, an 8-bit shift register consisting of two 74LS194 shift registers is utilized. The 2-to-1 MUX is used to control the input, specifically to choose the New Data In or the Previous data.

2 OPERATION OF THE MEMORY CIRCUIT

a. **Describe how the addressing is implemented. When does the circuit commit a read or write from/to the input switches and output register respectively?**

   To implement the addressing, a counter (74193) and a comparator (7485) are needed. The counter will work continuously to determine the relative address of the words. Then, the comparator will compare that address with the address that is placed in the Storage Address Register (SAR). The comparator gives output "1" when the addresses match.
   The circuit commits a write when the output of the comparator = "1" and STORE = "1". In this condition, data in the SBR will be stored in the specified address by SAR.
   The circuit commits a read when the output of the comparator = "1" and the FETCH = "1". In this condition, the data from the shift register, specified by the SAR, will be loaded into the SBR.

b. **Describe how a write operation is performed on the memory. Describe what switches you flip and in what order. Describe intuitively how the data flows through the circuit at each clock cycle.**

The required condition for the write operation to be performed on the memory is that when the comparator (7485) outputs "1" (i.e., the count of counter matches the address in SAR), STORE is "1", and the LDSBR is high.
Indicate the address of register to store into by using SAR switches. Then, make DIN (data word to be loaded into SBR for storing into storage) as an input to 3x1 MUX.
This data (i.e., DIN) will be loaded into the SBR by 3x1 MUX if LDSBR is high.
Before the value in SAR is matches to the value counter, the shift registers continuously maintain data inside it by utilized the 2x1 MUX.
Then, when the STORE is high and the counter value matches SAR, the 2x1 MUX will write the data in SBR to the specific address in the register determined by the SAR.

c. **Describe how a read operation is performed on the memory. Describe what switches you flip and in what order. Describe intuitively how the data flows through the circuit at each clock cycle.**

For the read operation, first, indicate the memory address of the word you want to read by setting the SAR.
Then, when FETCH and the comparator output are high (i.e., the counter matches the address in SAR), the data from the shift-register specified by SAR is loaded into the SBR through 3x1 MUX.
However, before the counter matches SAR, the 3x1 MUX will keep the original value of SBR stored in it by looping back data into SBR. Additionally, the shift registers continuously maintain data inside it by utilized the 2x1 MUX until the comparator outputs "1".

3 WRITTEN DESCRIPTION AND BLOCK DIAGRAM OF MEMORY CIRCUIT IMPLEMENTATION

3.1 WRITTEN DESCRIPTION OF MEMORY CIRCUIT IMPLEMENTATION

Addressing is a crucial part of Shift-Register Memory. We need to make sure that data is written or read to/from the correct register address as we desire. The **counter (using 74193: synchronous up/down 4-bit binary counter)** is used to increment the address with each clock cycle. The **comparator (using 7485: a 4-bit magnitude comparator of two binary format inputs)** is used to compare the counter's output with the address placed in the SAR, gives output "1" if the addresses match, indicates the correct register is selected.
To perform the storage function, two **shift-registers (using 74194: a 4-bit bidirectional universal shift register)** are utilized, one as bit 0 and the other as 1 bit. These shift-registers always shifts right.
A **2x1 Multiplexer (using 72157: Quad 2-input Multiplexer)** will work to determine whether new data from SBR (i.e., write operation) or previous data that is cyclically shifted should be fed into the register.
The other multiplexer, a **3x1 Multiplexer (using 74153: dual 4-to-1 line multiplexer)**, is directly connected to the D-flip flop and SBR, and its operation is controlled by the output signals of the comparator, FETCH, and LDSBR (as explained in part 2). For write operations, a 3x1 MUX is used to load the DIN into the SBR (i.e., buffer: a

temporary storage area for data before it is written to the register). Moreover, for read operations, a 3x1 MUX is used to load data from the register into the SBR.
A **D-flip flop (using 7474: a dual edge-triggered D-type flip-flop,)** is used to implement the SBR to store the data temporarily. It prevents data loss during the counter cycles. The input is controlled by the output of the 3x1 MUX, and the output of the D-flip flop is SBR.

3.2 HIGH LEVEL BLOCK DIAGRAM

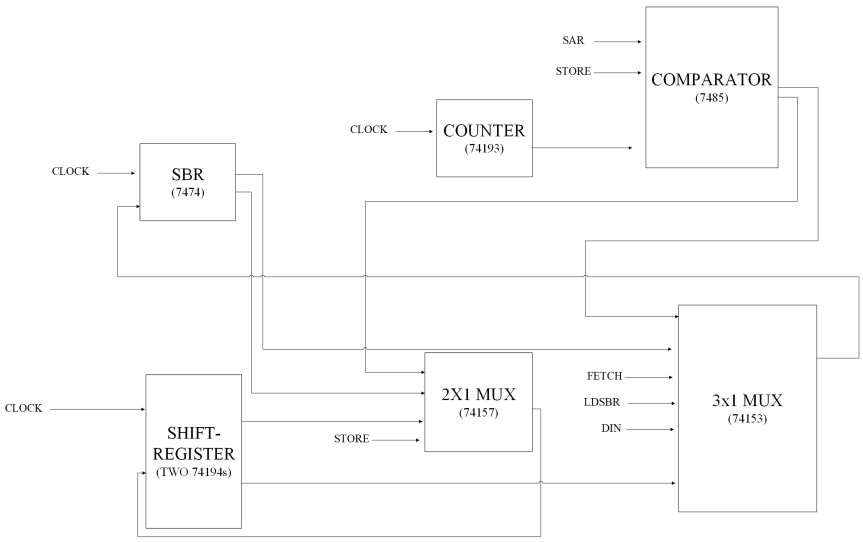

Figure 2: High Level Block Diagram

4 CONTROL UNIT

4.1 WRITTEN DESCRIPTION OF THE CONTROL UNIT

The control unit of this circuit contains the counter and comparator, which will determine the work of the 3x1 MUX and 2x1 MUX. The inputs are CLOCK, SAR, FETCH, STORE, and LDSBR.
- The clock controls the counter (74193) for addressing purposes.
- The counter's output and SAR control the comparator (7485); only when they are both the same does the comparator output "1."
- The comparator output, FETCH, and LDSBR control the 3x1 MUX (74153) in deciding the load path. The DIN data will be loaded to SBR only when the comparator output and the LDSBR are high; in addition, when the comparator output and FETCH are high, SBR will be loaded with data from the register.
- The comparator output and STORE control the 2x1 MUX (74157) in deciding which data will be loaded to the register, whether previous data or new data from SBR.

4.2 HIGH LEVEL BLOCK DIAGRAM OF THE CONTROL UNIT

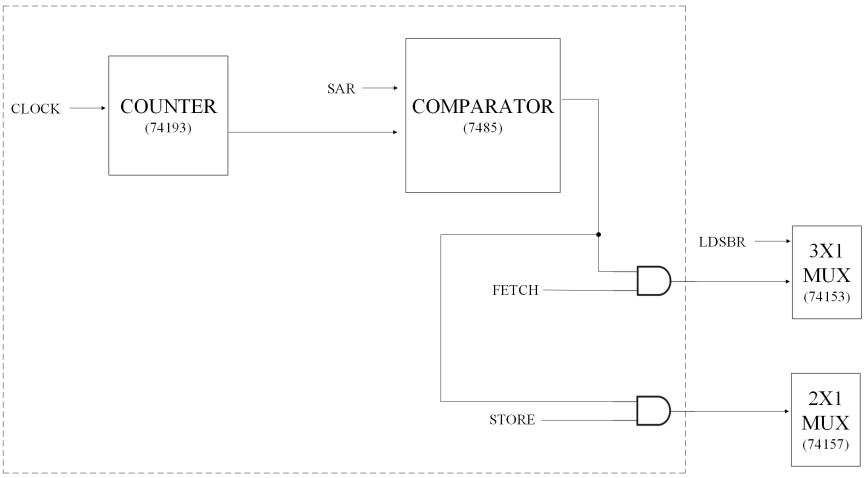

Figure 3: Control Unit Diagram

5 DESIGN STEPS TAKEN AND DETAILED CIRCUIT SCHEMATIC

5.1 KARNAUGH MAPS AND TRUTH TABLE

Together with another component, 3x1 MUX plays an important role in the built circuit. Below are the truth table and the k-map of 3x1 MUX.

COMPARATOR OUTPUT	STORE	FETCH	LDSBR	S1	S0
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
1	0	0	0	0	0
1	1	0	0	0	0
0	0	0	1	0	1
1	0	0	1	0	1
1	0	1	0	1	0

Figure 4: Truth Table of 3x1 MUX

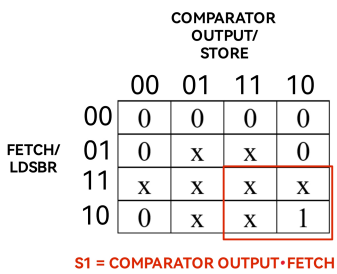

Figure 5: K-Map of S1

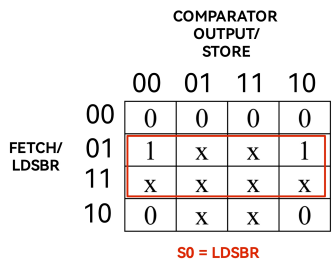

Figure 6: K-Map of S0

Based on the K-maps above, for 3x1 MUX, the Boolean expressions are S1 = COMPARATOR OUTPUT * FETCH and S0 = LDSBR. Meanwhile, trivially, the Boolean Expression for 2x1 MUX is S = COMPARATOR OUPUT * STORE.

5.2 WRITTEN DESCRIPTION OF THE DESIGN CONSIDERATIONS TAKEN

When building the circuit, we considered whether to use a synchronous (74169 or 74193) counter or a ripple (7493) counter. Each of these counters has certain advantages and disadvantages. The ripple counter is an asynchronous design, which means each clock is derived from the previous output. In addition, it is simple to build and use. However, there is a very large potential for a glitch to occur during the operation using this ripple counter.

On the other hand, the synchronous counter is a synchronous design, which means each flip-flop will get the same clocks. Unlike the ripple counter, the synchronous counter is a complex design because it is harder to extend to more bits. Nevertheless, this counter is able to remove the glitch in the output.

In the end, we used a 74193 synchronous counter since we try to avoid glitches as much as we can in order to make the entire circuit function well and ensure no undesired behavior occurs. In the implementation, the glitches may still exist, but this would not be a problem. The circuit is synchronized to the clock, so it can be ensured that changes will definitely occur at a predictable time. So, as long as the time constraint is achieved or fulfilled, the glitch will not affect the operation of the circuit.

5.3 DETAILED CIRCUIT SCHEMATIC

The complete circuit that is built by using Quartus is shown below.

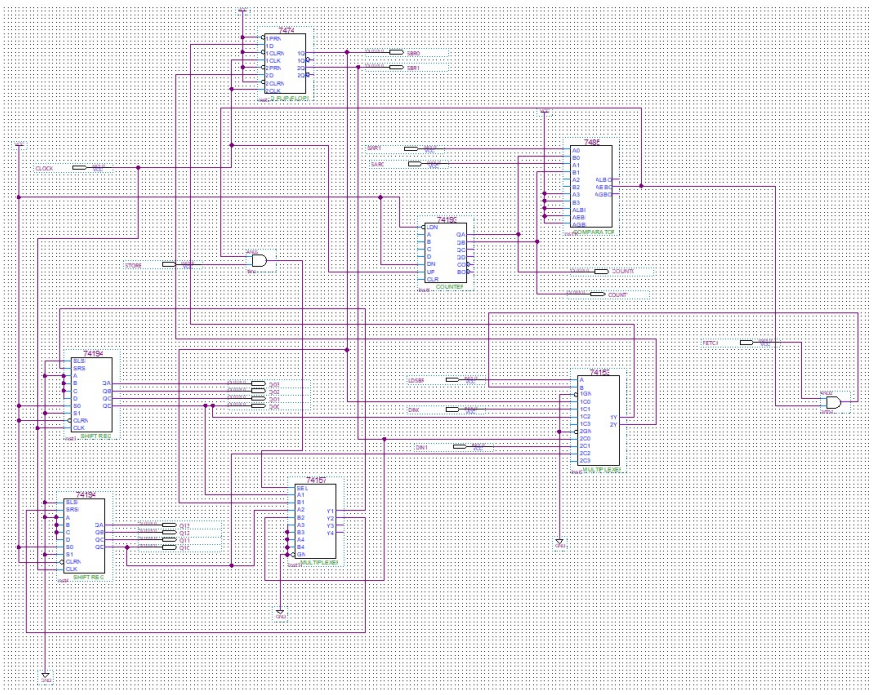

Figure 7: 2-bit, 4-Word Shift Register Storage Unit

The circuit above can be partitioned into several core parts, which are shown below.

a. **Shift Register and 2x1 Multiplexer**

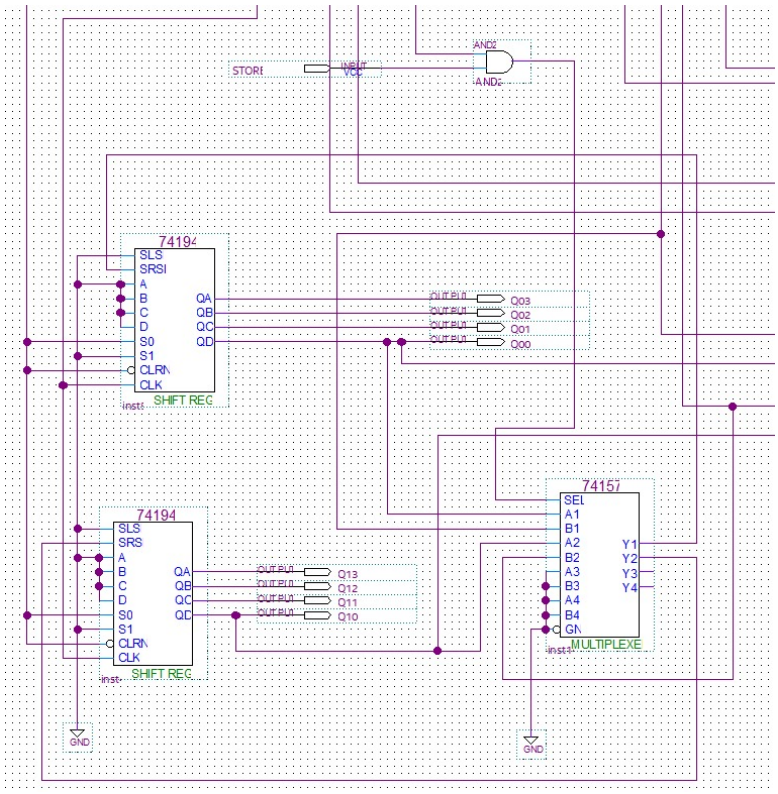

Figure 8: Shift Register and 2x1 MUX

b. **Control Logic for Addressing**

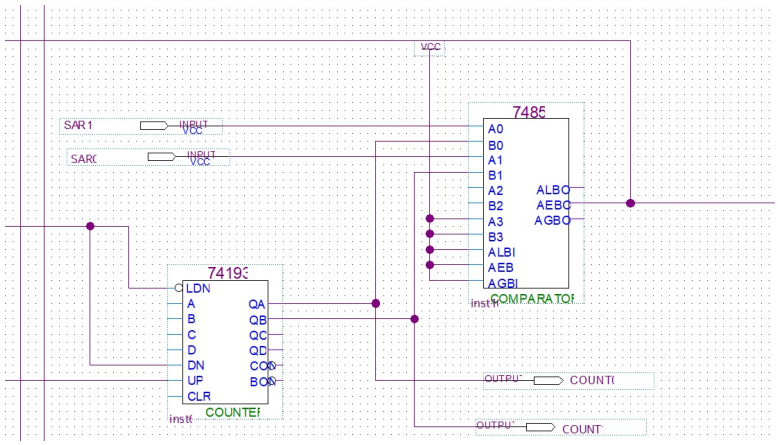

Figure 9: Control Logic for Addresing

c. **3x1 Multiplexer**

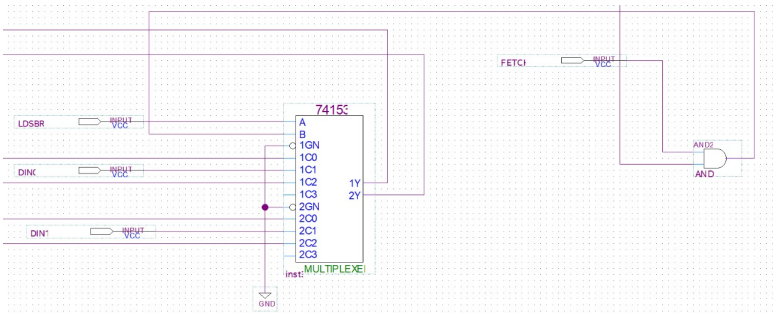

Figure 10: 3x1 MUX

d. **SBR (Storage Buffer Register)**

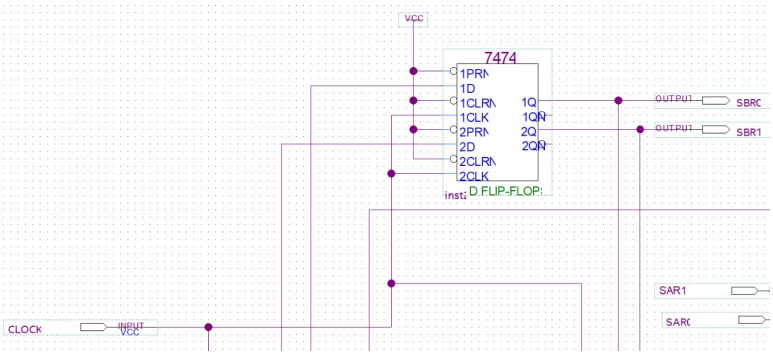

Figure 11: SBR

6 DESCRIPTION OF ALL BUGS ENCOUNTERED, AND CORRECTIVE MEASURES TAKEN

- The 7485 did not output "1" when SAR matched the counter, leading to an error in do read/write operation. This happened due to a misconnection between the QC output of the counter and the A1 input of the comparator. A1 is actually for input SAR, not the output of the counter. Thus, the incorrect behavior of the comparator is occurred. We re-examined the data sheet and the wiring of the circuit and found the misconnection. Then, we corrected it by disconnecting that connection and connecting A1 to SAR.
- The 74193 counter did not work correctly. The cause is that we accidentally connected the CLR pin to GND, which reset the counter. To overcome this problem, we disabled the connection of the CLR pin to the ground and left it unconnected.
- There is a problem in 3x1 MUX selection, leading to incorrect data in the write/read operation. This problem is due to improper input for 3x1 MUX. By carefully studying the data sheet and writing the truth table and K-map, we solved this problem.
- In addition, because we are not used to using Quartus, we encountered errors due to path errors or other technical errors several times that prevented the simulation from occurring.

7 CONCLUSION

7.1 SUMMARY

We design a simple 2-bit, 4-word shift-register storage unit. The four main operations of this shift-register memory are load, read, write, and do nothing. These operations are performed by using a 4-bit shift register (74194), counter(74193), comparator (7485), 3x1 multiplexer (74153), 2x1 multiplexer (74157) and D-flip-flop (7474). Then, the function of the circuit is controlled by several signals: LDSBR, FETCH, and STORE. Our designed circuit works correctly, meeting the goal.

7.2 ANSWERS TO ALL THE PRE-LAB QUESTION

a. **The clock must run continuously – do not gate the clock (this is bad practice in digital design, why?)**

   If the clock is gated, a timing issue may occur. Gating the clock can cause propagation delays, which can lead to race conditions or race hazards (i.e., the condition of digital design where the system's substantive behavior is dependent on the timing of other uncontrollable events, leading to unexpected or inconsistent results). In simpler terms, gating the clock can generate glitches. In this lab, the shift-registers have to shift data on each clock pulse continuously. If the clock is gated, the data may be inconsistently shifted, which violates the storage functionality of the circuit (or asynchronous problems).

b. **Only the clock input needs to be de-bounced in order to step through your circuit (why?)**

   Timing and synchronization are a really crucial thing in the circuit. When the clock signal is switched, there are possible noises that can be misinterpreted as multiple clock signals. By de-bouncing the clock input, we avoid the unwanted clock edges that may cause asynchronous or unexpected behavior from the system. Only the clock input needs to be de-bounced because the clock signal plays an important and primary role in controlling the timing of the system.

7.3 ANSWERS TO ALL THE POST-LAB QUESTION

a. **What are the performance implications of your shift register memory as compared to a standard SRAM of the same size?** There are some differences between shift register memory built in this lab and the standard SRAM of the same size:
   - SRAM is faster than shift register memory because it provides direct access to data, allowing read and write operations to happen immediately. On the other hand, shift registers require sequential shifting, which can cause delays. Hence, the shift register is slower than SRAM.
   - SRAM is more complex in design than shift registers and requires more space. The difference in the number of components used to build SRAM and shift registers makes shift registers cheaper to form.
   - SRAM consumes more power than shift registers due to the requirement to maintain the stability of its components.

b. **What are the implications of the different counters and shift register chips, what was your reasoning in choosing the parts you did?**

We have to choose between a synchronous counter or a ripple counter. Finally, we chose the synchronous counter (74193) since the ripple counter has potential glitches in operation, whereas the synchronous counter removes the possibility of glitches in outputs. We want to avoid the glitch that may lead to incorrect behavior of our circuit because counter 74193 is synchronous. For shift register chips, two 74194 shift registers are used. It is a 4-bit universal shift register. We use two 74194 chips because the aim is to build a 2-bit, four-word shift-register storage unit. The multiplexer is connected to the SRSI and SLSI to the GND since only the leftmost bit is needed to write. To make it always shift right, S1 is low (i.e., is connected to GND), and S0 is high (i.e., is connected to VCC). Even though at the beginning, we also considered the 7495, a 4-bit parallel-access shift register. However, we chose the 74194 over the 7495 because the 74194 shift register, SBR, and counter have the same clock cycle.